CSS4005 Final Project — Production-Grade Cloud Platform

Weight: 40% of the course grade (the "Final Cloud Infrastructure Project / Project with defence" in the syllabus). **Work mode:** individual. Everyone builds their own platform from their own personal **variant**. **Deadline:** any day before the final exam, **including the final exam day**. Present and defend whenever you're ready. **Result:** a public GitHub repository you can show an employer, plus a report, a short presentation and an oral defense.

---

1. Overview

You get a small, ready-made web application: `campus-app` from the template repository `weeebdev-edu/css4005-final-project`. The app is not what we grade. What we grade is the **platform you build around it**: the network, compute, database, storage, identity, secrets, monitoring, deployment pipeline and the evidence that it survives failures and stays secure and affordable.

Everything you practiced this semester comes together here. By the end you should be able to say: *"I designed, built, secured, monitored and operated this platform myself, and I can rebuild it from code in minutes."*

What the template app gives you

Feature	Detail
Framework	FastAPI, configured only through environment variables (`APP_NAME` = your `theme`)
Database	PostgreSQL or MySQL via `DB_URL` ; optional ClickHouse analytics via `CLICKHOUSE_URL`
Object storage	`STORAGE_BACKEND=local|s3|gcs` for attachments
Health	`/healthz` (process alive), `/readyz` (DB and storage reachable)
Metrics	`/metrics` in Prometheus format (request count, latency, errors)
Logs	JSON lines on stdout with a `request_id` on every line
Secrets	every secret can be read from a file: `DB_PASSWORD_FILE` , `SECRET_KEY_FILE` , … `SECRET_KEY` must be ≥ 32 characters or the app refuses to start
Security helper	`POST /api/admin/check-password` validates a password against a policy
SQL	example role scripts for `admin` , `app_rw` , `app_ro`
Container	non-root `Dockerfile` , `compose.yaml` for local development, a minimal CI workflow

You may change the app (add endpoints, metrics, fix bugs). You may **not** replace it with a different app.

---

2. Learning goals ↔ syllabus

Week	Topic	Where it appears in the project
1	Cloud computing & AWS/GCP intro	Track choice, region choice, shared-responsibility section of the report
2	Linux for cloud	Hardened VMs, startup scripts, systemd services, backup scripts
3	Compute (EC2 / Compute Engine)	Launch/instance templates, user data, VM fleet
4	Networking (VPC)	3-tier network across 2 zones, SG / firewall chaining, NACL or layered firewall
5	Storage (EBS + S3 / PD + GCS)	DB data disk + snapshots, bucket with versioning, lifecycle, Block Public Access, presigned URLs
6	IAM & cloud security	Least-privilege roles / service accounts, no keys on servers
7	Databases	**Self-hosted** primary + replica, roles, backups, restore test, comparison with managed DB
8	Architecture	Diagram, Well-Architected review, Architecture Decision Records (ADRs)
9	Load balancing & auto scaling	LB in ≥ 2 zones, health checks, scaling policy, self-healing
10	Docker & containers	App image built, scanned, pushed to ECR / Artifact Registry; the app tier runs on a **Kubernetes cluster** across 2 zones
11	Infrastructure as Code (Terraform)	Everything in Terraform modules; reproducible `apply` / `destroy`
12	Monitoring & logging	Dashboards, ≥ 4 alerts, centralized logs, request tracing by `request_id`
13	CI/CD	GitHub Actions: lint → test → build → scan → push → deploy, with rollback
14	HA, security, cost	Zone-failure drill, DB failover drill, security findings, cost estimate vs actual

---

3. Tracks: AWS or GCP

Choose **one** track. The requirements, the rubric and the points are **identical**. Only the service names change.

- **AWS track:** your own AWS account (free tier / credits) **or** the **AWS Academy Learner Lab**.
- **GCP track:** your own Google Cloud account (free trial credits).

Not sure what you can afford? Read the **Free Tier Guide** first: what is free on each cloud, what costs money in this project, and how to keep the whole build inside your budget.

AWS ↔ GCP mapping

Area	AWS	GCP
Network	VPC, subnets, route tables, IGW, NAT Gateway	VPC network, subnets, routes, Cloud NAT
Network security	Security Groups (stateful, can reference other SGs), NACLs (stateless)	Firewall rules with **network tags** or **service accounts** as source/target; hierarchical/VPC firewall policies for layering
Compute template	Launch Template + user data	Instance Template + startup script
VM fleet	Auto Scaling Group (ASG)	Managed Instance Group (MIG) with autoscaler
Load balancer	Application Load Balancer (ALB) + target groups	External Application (HTTP(S)) Load Balancer + backend service + health check
Disks & snapshots	EBS volumes, EBS snapshots, Data Lifecycle Manager	Persistent Disk, snapshots, snapshot schedules
Object storage	S3 (versioning, lifecycle, Block Public Access, presigned URLs)	Cloud Storage (versioning, lifecycle, public access prevention, signed URLs)
Identity	IAM roles + instance profiles	Service accounts attached to VMs
Secrets	Secrets Manager (or SSM Parameter Store `SecureString`)	Secret Manager
Container registry	Amazon ECR (scan on push)	Artifact Registry (vulnerability scanning)
Kubernetes (self-managed)	k3s on EC2 nodes (ASG)	k3s on Compute Engine nodes (MIG)
Kubernetes (managed, if you have credits)	Amazon EKS	Google Kubernetes Engine (GKE)
Kubernetes behind the cloud LB	ALB → NodePort / k3s ingress (Traefik) on the nodes, or AWS Load Balancer Controller	HTTP(S) LB → NodePort / ingress on the nodes, or GKE Ingress
Monitoring	CloudWatch metrics, dashboards, alarms; CloudWatch agent	Cloud Monitoring dashboards, alerting policies; Ops Agent
Logging	CloudWatch Logs + Logs Insights	Cloud Logging + Logs Explorer

Area	AWS	GCP
Certificates	ACM (or self-signed imported for the lab)	Google-managed or self-managed SSL certificates
Remote access without SSH keys	SSM Session Manager	IAP TCP forwarding
Infrastructure as Code	Terraform `hashicorp/aws` provider	Terraform `hashicorp/google` provider
Cost tools	AWS Pricing Calculator, Cost Explorer, Budgets	Google Cloud Pricing Calculator, Billing reports, budgets

Learner Lab notes (AWS track)

Per the Learner Lab Readme, the lab typically allows only **`us-east-1`** (and `us-west-2`). You **cannot create IAM users or IAM roles**: use the provided **`LabRole`** and **`LabInstanceProfile`**. Lab sessions expire after a few hours and credits are limited. That is fine: your Terraform code must rebuild everything quickly, and you must explain in the report which least-privilege role **you would have created** in a real account (write the policy JSON) even though the lab forced you to use `LabRole`. If a service is blocked in the lab (for example Secrets Manager), use the documented fallback (SSM Parameter Store `SecureString`) and say so in the report.

---

4. Your personal variant

Every student gets a **personal variant** from the instructor. It is generated from your student ID, and your platform **must** follow it. **See your variant from the instructor.** If you didn't get one, ask before you start.

Your variant is one row with these fields:

Field	Possible values	Where it must be visible
`theme`	a domain, e.g. `library`, `dorm-repairs`, `club-events`	`APP_NAME` and the app's items (what the CRUD stores)
`resource_prefix`	e.g. `ci-nurlanov-4821`	**Every** resource name and a `Project` tag/label
`vpc_cidr`	`10.X.0.0/16`	Your VPC and all subnet CIDRs carved from it
`traffic_pattern`	`steady`, `spiky`, `scheduled-peaks`	Scaling policy type and the load test you run
`rpo`	`15 min`, `1 h`, `4 h`	Backup / snapshot schedule and replication must meet it
`rto`	`30 min`, `1 h`, `4 h`	Restore and failover drills must finish within it
`monthly_budget_usd`	`15`, `25`, `40`	A **budget alert at this cap**; instance sizes and NAT choices
`db_engine`	`postgresql`, `mysql`	Your self-hosted primary + replica
`clickhouse_required`	`yes`, `no`	If `yes`: a self-hosted ClickHouse node used via `CLICKHOUSE_URL`
`cluster`	`k3s-on-autoscaled-vms`, `k3s-fixed-3-nodes`, `managed-k8s-if-credits`	How your Kubernetes cluster is built (see §5.4)
`hpa_cpu_target`	`50`, `60`, `70`	The CPU target of your HorizontalPodAutoscaler
`password_min_length`	`12` – `16`	DB password policy, app checker, CI password test
`password_require_symbol`	`yes`, `no`	Same three places
`alert_cpu_pct`	`60`, `70`, `80`	CPU alarm threshold (and a sensible scaling target near it)
`alert_5xx_per_min`	`5`, `10`, `20`	5xx alarm threshold
`backup_retention_days`	`7`, `14`, `30`	Snapshot and logical-backup retention (lifecycle + schedule)

You must **show that each value is applied**: the prefix in resource names, the CIDR in the network, the cluster type, the HPA target, the scaling policy and alarm thresholds, a backup schedule that meets your RPO, drills that meet your RTO, the password policy rejecting weak passwords, and a budget alert set at your cap. The report has a table for this.

Ignoring the variant caps the report score (see RUBRIC.md). Two identical platforms from two students are treated as plagiarism.

---

5. Required components and acceptance checklists

Each block below lists what you must build and what you must **show** in the report. "Evidence" means a screenshot, command output, Terraform code reference or log excerpt, labelled with your resource prefix.

5.1 Networking (week 4)

□ One VPC with **your variant CIDR**, spanning **2 availability zones** (GCP: 2 zones in one region).
□ Three subnet tiers per zone: **public** (LB, NAT), **app** (private), **db** (private). 6 subnets on AWS; on GCP, subnets are regional, so use **3 regional subnets** and show the zones your instances use.
□ Egress for private tiers via NAT (NAT Gateway / Cloud NAT). Budget tip: one NAT for both zones is acceptable if you explain the trade-off in an ADR.
□ **Security chaining:** LB ← internet on 80/443 only; app ← LB only; DB ← app only (by SG reference / network tag / service account, **not** by CIDR to the whole VPC).
□ A **second layer**: NACLs on the DB subnets (AWS) or a deny-by-default firewall policy with explicit allows (GCP).
□ No `0.0.0.0/0` inbound anywhere except the LB on 80/443.
□ Evidence: network diagram with CIDRs, route tables, SG/firewall table, and a test showing the DB port is **unreachable** from the internet and from the public tier.

5.2 Compute and Linux (weeks 2–3)

□ **Launch templates / instance templates** with startup scripts for every VM role: Kubernetes nodes (§5.4) and DB servers (§5.6). A VM must come up fully configured with no manual steps.
□ Hardened Linux: services run as **non-root** users (systemd units, container `USER`), automatic security updates or a documented patching approach, unnecessary ports closed.
□ **No SSH from the internet**: SSM Session Manager (AWS) or IAP (GCP); or SSH only from your own IP via a bastion, documented.
□ No access keys or passwords in user data / startup scripts.
□ Evidence: template definitions, the startup scripts, `systemctl status` on a node and a DB server, `ss -tulpn` output.

5.3 Storage (week 5)

□ An object-storage bucket (S3 / GCS) named with your prefix, with **versioning**, a **lifecycle rule** (expire noncurrent versions, move old objects to a cheaper class), and **Block Public Access / public access prevention** on.
□ **Encryption at rest** on the bucket and on all disks.
□ App attachments stored in the bucket and downloaded through **presigned / signed URLs** with a short expiry.
□ The DB data lives on a **separate persistent disk** (EBS / PD) from the OS disk.
□ **Scheduled snapshots** of the DB data disk with your variant's retention (Data Lifecycle Manager / snapshot schedule or a script).

□ Evidence: bucket settings, lifecycle JSON, a presigned URL working and expiring, snapshot list with dates.

5.4 Kubernetes cluster for the app tier (weeks 9–10)

The **app tier runs on a Kubernetes cluster** spread over **≥ 2 zones**, in the **app** subnets. The **database does NOT go into the cluster**: it stays self-hosted on VMs in the db subnets (§5.6).

Your variant's `cluster` field decides how you build it:

- **`k3s-on-autoscaled-vms`:** a k3s server node plus agent nodes; the agents are an **ASG / MIG** (min ≥ 2, across 2 zones) built from a launch / instance template that joins the cluster at boot (join token read from the secret store, never hard-coded).
- **`k3s-fixed-3-nodes`:** three k3s VMs (one server + two agents or a 3-server HA setup) spread over 2 zones, each created from a template by Terraform. Scaling happens at the pod level (HPA); node replacement is done by re-applying Terraform.
- **`managed-k8s-if-credits`:** EKS or GKE with a node group / node pool over 2 zones. Only if your account has the credits; Learner Lab students with this variant may fall back to `k3s-on-autoscaled-vms` and say so in the report.

Required in every case (manifests or Helm chart in your repo):

□ The app image comes from ECR / Artifact Registry (pulled with the node role / service account or an image-pull secret built at boot, never committed).
□ A **Deployment** with ≥ 2 replicas, **resource requests and limits**, a **liveness probe on `/healthz`** and a **readiness probe on `/readyz`**, running as a non-root user.
□ Pods spread over both zones (`topologySpreadConstraints` or pod anti-affinity on the zone label).
□ A **Service** and an **Ingress** (or a `LoadBalancer` Service), **fronted by the cloud load balancer** (§5.5).
□ Configuration in a **ConfigMap**; secrets in a Kubernetes **Secret** created at deploy time from the **cloud secret manager** (§5.7), never committed to git.
□ A **HorizontalPodAutoscaler** with your variant's **`hpa_cpu_target`**, min ≥ 2.
□ A **rolling update** (new image tag) with zero failed requests during the rollout, and a **rollback** (`kubectl rollout undo` or redeploying the previous tag).
□ A **node / zone failure drill**: `kubectl cordon` + `drain` a node, or stop a node VM, and show the pods rescheduled on the remaining nodes while the site keeps answering.
□ Evidence: `kubectl get nodes -L topology.kubernetes.io/zone`, `kubectl get pods -o wide`, `kubectl get hpa`, `kubectl rollout history`, and the drill output.

5.5 Load balancing and auto scaling (week 9)

□ A cloud **load balancer** (ALB / HTTP(S) LB) in **≥ 2 zones**, forwarding to the cluster (NodePort or the k3s ingress on the nodes, or a controller-managed LB on EKS/GKE). Health check on the app's **`/readyz`** through the ingress.
□ Two levels of scaling that fit your **`traffic_pattern`**: - **pods:** the HPA (§5.4); - **nodes:** the ASG / MIG scaling policy (`k3s-on-autoscaled-vms`), the managed node group / cluster autoscaler (`managed-k8s-if-credits`), or a documented capacity plan (`k3s-fixed-3-nodes`). Use target

tracking for `spiky`, scheduled actions + target tracking for `scheduled-peaks`, a modest target for `steady`.
□ **Load test** (k6, hey, ab, locust…) showing pods (and nodes, where applicable) scaling out and back in, with graphs.
□ **Self-healing demo:** kill a pod and terminate a node VM → the LB keeps serving → Kubernetes and the group replace them.
□ HTTPS on the LB: ACM / Google-managed cert if you have a domain, otherwise a self-signed certificate (document the browser warning and why a real domain fixes it).

5.6 Self-hosted database (week 7) — no RDS / Cloud SQL

□ Your variant engine (**PostgreSQL or MySQL**) installed on VMs in the **db** subnets: **one primary + one replica** in **different zones**, using streaming (PostgreSQL) or async binlog (MySQL) replication.
□ If your variant requires it: a **ClickHouse** node in the db tier, wired to the app via `CLICKHOUSE_URL`.
□ Roles: `admin` (schema owner, not used by the app), `app_rw` (DML only on the app schema), `app_ro` (read-only, for reports / the replica). Show a denied action for each limited role.
□ **Password policy enforced in the database** with your variant's parameters: PostgreSQL `passwordcheck` / `credcheck` (or an equivalent enforced path), MySQL `validate_password`. Show a weak password being **rejected**. Also use the app's `/api/admin/check-password`.
□ **Backups:** scheduled snapshots (§5.3) **and** logical backups (`pg_dump` / `mysqldump`) uploaded to the bucket on a schedule.
□ **Restore test:** restore a logical backup into a fresh database and prove the data is there; measure how long it took against your RTO.
□ **Failover drill:** stop the primary, promote the replica, point the app to it (manually or by script), measure downtime against RTO.
□ DB reachable **only** from the app tier.
□ **Report section:** compare your self-hosted DB with RDS / Cloud SQL (backups, patching, failover, cost, effort). When would you choose each?

5.7 Identity (IAM) and secrets (week 6)

□ Instances access the bucket, registry and secrets through an **instance role / service account**, never access keys.
□ **Least privilege:** the app's role/service account can only read its own secrets, read/write its own bucket prefix and pull its own image. (Learner Lab: you must use `LabRole`; write the least-privilege policy JSON you *would* attach and explain the gap.)
□ **Secrets in the cloud secret manager:** DB passwords (`admin`, `app_rw`, `app_ro`, replication user), `SECRET_KEY` (≥ 32 chars) and any other credential live in **Secrets Manager** (fallback: SSM Parameter Store `SecureString`) or **GCP Secret Manager**.
□ Instances fetch secrets **at boot** using their role and write them to `*_FILE` paths readable only by the app user (mode `0400`).
□ **Nothing secret** in git, user data, Docker images, Terraform variables files committed to git or Terraform **outputs** (mark them `sensitive = true`; generate passwords with `random_password` and know that state still contains them: store state safely and say how).
□ A **rotation plan**: how you would rotate the `app_rw` password without downtime (and do it once as a stretch).

5.8 Architecture (week 8)

□ An **architecture diagram** with zones, subnets, CIDRs, LB, groups, DB primary/replica, bucket, registry, secret store, monitoring.
□ A **Well-Architected review**: one paragraph per pillar (operational excellence, security, reliability, performance efficiency, cost optimization, sustainability) with concrete evidence from your build.
□ **At least 4 ADRs** (Architecture Decision Records: context → decision → consequences), e.g. one NAT vs two, self-hosted DB replication mode, cluster type and ingress path, TLS approach.

5.9 Containers and CI/CD (weeks 10, 13)

□ GitHub Actions workflow on every push/PR: **lint → test → build image → security scans → push** to ECR / Artifact Registry with an immutable tag (git SHA).
□ Deploy to the cluster on merge to `main`: update the Deployment's image tag (`kubectl set image` / `kubectl apply` / `helm upgrade`), wait with `kubectl rollout status`, and fail the job if readiness probes don't pass.
□ **Rollback** to the previous image tag, demonstrated once.
□ CI credentials: GitHub OIDC → cloud role / Workload Identity Federation **if your account allows it**; with Learner Lab, short-lived lab credentials in GitHub Secrets or a self-hosted runner on an instance with `LabInstanceProfile`. Document what you did and why.

5.10 Security checks (weeks 6, 14)

All checks run in CI **and** their results go into the report's **security findings table** (finding → severity → fixed / accepted + why).

□ **Secret scanning:** gitleaks on the whole git history; the build fails on findings.
□ **Image scanning:** Trivy (and ECR / Artifact Registry scan); fail on CRITICAL.
□ **IaC scanning:** Trivy config, tfsec or Checkov on your Terraform.
□ **Password policy checks:** DB-level enforcement (§5.6) + the app checker + a CI test that the generated passwords satisfy your variant's policy.
□ **Network review:** a table of every SG/firewall rule with justification; no `0.0.0.0/0` inbound except LB 80/443.
□ **Storage review:** no public buckets, encryption at rest everywhere.
□ **IAM review:** what each role/service account can do and why.
□ **TLS** on the LB (§5.5).

5.11 Infrastructure as Code (week 11)

□ **Everything** above is in Terraform, organised in **modules** (network, cluster/compute, database, storage, security/IAM, monitoring). Kubernetes objects live as manifests / a Helm chart in the repo and are applied by CI (or by Terraform's kubernetes/helm providers).
□ Remote or otherwise protected state (S3 + locking / GCS backend, or a documented safe local state that is **not** committed).
□ `terraform fmt -check` and `terraform validate` in CI.
□ **Reproducibility:** `terraform destroy` then `terraform apply` rebuilds a working platform. Record the time it takes.
□ Your **variant values** live in one `terraform.tfvars` (no secrets in it).

5.12 Monitoring and logging (week 12)

□ **Dashboards** with at least: LB requests / 5xx / latency; app RED metrics (Rate, Errors, Duration) from `/metrics` (CloudWatch agent Prometheus scraping / Ops Agent / your own Prometheus in the cluster); node CPU and memory; pod restarts and HPA replica count; DB metrics (replication lag, connections, disk).
□ **≥ 4 alerts**, including **CPU above** `alert_cpu_pct` and **5xx above** `alert_5xx_per_min` from your variant, plus at least two of: unhealthy targets, p95 latency, replication lag, disk usage. Show one alert **firing** and **resolving** (email/SNS/notification channel).
□ **Centralized logs:** app (pod) JSON logs and DB logs shipped to CloudWatch Logs / Cloud Logging (Fluent Bit DaemonSet, CloudWatch agent or Ops Agent).
□ A **saved query** that follows **one** `request_id` from the LB log (if enabled) through the app logs.

5.13 High availability, security and cost (week 14)

□ **Zone-failure drill:** take one zone out (stop or drain all cluster nodes in it, remove its subnet from the node group) → pods reschedule to the other zone and the platform keeps serving; record user-visible impact.
□ **DB failover drill** (§5.6) with measured RTO and data loss vs RPO.
□ **Cost estimate** in the AWS / Google Cloud Pricing Calculator for running 24/7 for a month, **actual spend** from the billing console for your project period, and **at least 3 cost optimizations** you applied or would apply (rightsizing, scheduled scale-in, single NAT, lifecycle to cold storage…). Set a **budget alert at** `monthly_budget_usd` and stay within it (Learner Lab: show the lab's spend counter instead, since Budgets aren't available).
□ **Teardown proof:** after grading, `terraform destroy` and a screenshot showing no resources left.

---

6. Suggested milestones (optional)

You decide your own pace. A realistic plan:

Milestone	Content
M1	Fork the template, run it locally with `compose`, read your variant, draw the diagram
M2	Terraform network + bastion-less access + bucket + secrets
M3	Self-hosted DB primary + replica, roles, password policy, backups
M4	Image in registry, Kubernetes cluster over 2 zones, Deployment + HPA, cloud LB + HTTPS
M5	CI/CD with scans, rolling deploy to the cluster, rollback
M6	Monitoring, alerts, logs, load test
M7	Drills (zone, DB failover, restore), cost, report, rehearsal

---

7. Submission and defense

What you submit (by the deadline you choose)

1. **Public GitHub repository** (created from the template) containing: app, Terraform modules, startup scripts, Kubernetes manifests / Helm chart, CI workflows, SQL role scripts, dashboards/alert definitions (as code), ADRs, and a **portfolio-quality** `README.md` (what it is, diagram, how to deploy, what you learned).
2. **REPORT.pdf** built from `REPORT_TEMPLATE.md`. It is graded with `RUBRIC.md`.
3. If your resources are already torn down by the defense: a **demo video** (≤ 10 min) showing the live platform, the drills and the alerts.

Presentation and defense

- **Presentation (~7 min):** the architecture, your variant, the hardest problem you solved, one drill.
- **Live demo:** either your running platform or a fresh `terraform apply` (start it before the defense) or the demo video.
- **Oral defense (~8 min):** questions about **your** code, **your** decisions and the course topics. You must be able to explain every file in your repository.

Grading summary (40 points)

Part	Points
Report (see RUBRIC.md)	20
Presentation & demo	8
Oral defense	12

---

8. Academic honesty

- The work is **individual**. You may discuss ideas; you may not share code, Terraform or report text.
- Your variant makes every platform different. A platform that doesn't match your variant, or matches another student's, is reviewed as plagiarism.
- AI assistants are allowed as tools, but **you** answer the defense questions. If you cannot explain a file, it doesn't count.
- Cite any external module, blog post or tutorial you used in the report.

---

9. FAQ

Will this cost money? It can. Load balancers, NAT gateways, VMs and managed Kubernetes control planes bill per hour. Read the **Free Tier Guide** before you build. Use your variant's budget cap as a hard limit: build, gather evidence, then `terraform destroy`. Rebuilding with Terraform is part of the skill. Set a budget alert on day one.

My variant says `managed-k8s-if-credits` but I don't have credits. Build `k3s-on-autoscaled-vms` instead and explain why in an ADR. It is graded the same.

I'm on Learner Lab and the session expired. Your resources may be stopped. Start the lab and re-run `terraform apply`. This is exactly why everything must be in code. Save your evidence (screenshots, logs, outputs) as you go.

Learner Lab blocks something I need (an IAM role, Secrets Manager, a service). Use `LabRole` / `LabInstanceProfile`, use the fallback (SSM Parameter Store `SecureString`), and document the gap and the real-account design in the report. You don't lose points for a lab limitation you documented properly.

Can I use RDS or Cloud SQL to save time? No. The database must be self-hosted on your VMs. You **compare** it with the managed service in the report instead.

Can I run the database inside Kubernetes (StatefulSet)? Not in this course. The database stays on VMs in the db subnets so that you practice disks, snapshots, replication and network isolation directly.

My resources must be torn down before the defense. Bring the demo video, your Terraform code and the evidence in the report. Be ready to run `terraform apply` live if asked and the budget allows.

Can I add a CDN, a WAF, a service mesh…? Yes, as extras, documented with an ADR, after all required components work. Extras don't replace required items.

Can I change the app? Yes: fix, extend, add metrics. You can't swap it for another app.